TechGuru AI Chatbot

Complete Developer Reference Guide

Azure AI Foundry • Python • Streamlit • Git

Built by: Muhammad Ali | April 2026
Live App: https://techguru.streamlit.app

SECTION 1 — WHAT WE BUILT

Project Overview

TechGuru is a domain-restricted AI chatbot built on Azure AI Foundry. It teaches technology, helps developers when stuck, and fetches live tech news using Bing Search grounding. It refuses all non-tech questions.

Core Features

- Learning Mode — gives structured roadmaps for any technology (Beginner → Intermediate → Advanced)
- Mentor Mode — answers developer questions with code examples and explanations
- Live News Mode — searches Bing in real-time for latest tech news
- Domain Restriction — politely refuses any non-technology questions

Technology Stack

Azure AI Foundry	Cloud platform hosting our AI agent, model, and Bing connection
GPT-4o-mini	The AI model that powers our chatbot responses

Bing Grounding	Connected search tool for real-time internet news
Python	Primary programming language for all backend logic
Streamlit	Python library that builds the chat web interface
OpenAI SDK	Used on cloud to call Azure /openai/v1 endpoint with API key
azure-ai-projects	Used locally to manage agents, connections, and Bing tools
Git + GitHub	Version control and code hosting
Streamlit Cloud	Free hosting platform for the live app

SECTION 2 — AZURE AI FOUNDRY SETUP

Azure AI Foundry Setup

Key Concepts

Azure AI Foundry	Microsoft's all-in-one AI platform. Use New Foundry (toggle ON) for latest features.
Project	A workspace inside Foundry. Holds your model, agent, and connections.
Agent	A configured AI worker with a name, model, system prompt, and tools.
Bing Grounding	A connection that lets your agent search the internet in real-time.
API Key	A secret key to authenticate API calls (limited permissions).
Entra ID	Microsoft's identity system. Required for full SDK management operations.
DefaultAzureCredential	Uses your local az login session. Works locally only.

Step-by-Step Azure Setup

1. Go to https://ai.azure.com and sign in
2. Click 'Create new' → Choose 'Microsoft Foundry Resource' (Recommended)
3. Fill in project name, region, subscription → Click Create
4. Once created, go to Overview page → Copy the 'Microsoft Foundry project endpoint'
5. Go to Overview → API Key section → Copy the key
6. Go to Agents section (New Foundry toggle ON) → Verify your project is ready

Finding Your Credentials

Project Endpoint	ai.azure.com → Your Project → Overview → 'Microsoft Foundry project endpoint'
API Key	ai.azure.com → Your Project → Overview → 'API Key' section → click eye icon
Bing Connection Name	ai.azure.com → Management Centre → Connected Resources → note the Name column
Model Name	ai.azure.com → Model Catalog → your deployed model name (e.g. gpt-4o-mini)

⚠ API Key vs Entra ID: The azure-ai-projects SDK ONLY supports Entra ID (az login). API Key only works for the /openai/v1 endpoint. This is why we use two different approaches for local and cloud.

Creating a Permanent Agent (Run Once Locally)

Run create_permanent_agent.py locally to create the TechGuru agent on Azure. This agent persists and is reused by the cloud app.

```
python create_permanent_agent.py
```

After running, verify at: ai.azure.com → Agents (New Foundry ON) → TechGuru should appear

SECTION 3 — LOCAL DEVELOPMENT SETUP

Local Development Setup

Prerequisites

- Python 3.9 or later — check with: python --version
- Git installed — check with: git --version
- Azure CLI installed — download from: https://aka.ms/installazurecliwindows
- VS Code (recommended editor)

Step 1 — Azure CLI Login

This is required for local development. It gives DefaultAzureCredential your identity.

```
az login
```

This opens a browser window. Sign in with your Azure account. Do this once — it persists.

💡 **You must re-run az login if your token expires (usually after a few hours of inactivity).**

Step 2 — Project Setup

```
mkdir techguru-chatbot
cd techguru-chatbot
python -m venv .venv
.venv\Scripts\activate    # Windows
source .venv/bin/activate   # Mac/Linux
```

Step 3 — Install Dependencies

```
pip install azure-ai-projects azure-identity openai python-dotenv streamlit requests
```

Step 4 — Create .env File

Create a file named .env in your project root. NEVER commit this to GitHub.

```
AZURE_ENDPOINT=https://your-resource.services.ai.azure.com/api/projects/your-project
MODEL_DEPLOYMENT_NAME=gpt-4o-mini
BING_CONNECTION_NAME=Bingsearch
# DO NOT add AZURE_API_KEY here — local uses az login instead
```

💡 ⚠ **Never put AZURE_API_KEY in your local .env. Local uses az login (DefaultAzureCredential). API key is only for Streamlit Cloud secrets.**

Step 5 — Run Locally

```
streamlit run main_ui.py
```

Opens at http://localhost:8501 in your browser automatically.

SECTION 4 — PROJECT STRUCTURE EXPLAINED

Project Structure

```
techguru-chatbot/
├── .env                      ← Secret keys (NEVER commit to GitHub)
├── .gitignore                ← Files Git should ignore
├── requirements.txt          ← All Python packages needed
├── main_ui.py                ← Streamlit UI — what user sees
├── create_permanent_agent.py ← Run once locally to create Azure agent
```

```
└── src/
    ├── paths.py              ← All file paths in one place
    ├── conn/
    │   ├── config.py         ← Loads all settings and secrets
    │   └── agent_instruction.txt  ← System prompt for the bot
    └── services/
        └── azure_services.py ← All Azure logic and TechAgent class
```

Why This Structure?

paths.py	Single source of truth for file paths. If you move files, update only here.
config.py	Reads secrets from Streamlit secrets (cloud) or .env file (local). One place for all config.
agent_instruction.txt	System prompt in a separate file. Easy to update without touching Python code.
azure_services.py	All Azure logic isolated here. UI code never deals with Azure directly.
main_ui.py	Pure UI code only. Calls azure_services.py for AI responses.

SECTION 5 — CODE EXPLAINED LINE BY LINE

Code Deep Dive

config.py — How Secrets Are Loaded

The get_config function tries three places in order:

```
def get_config(key: str) -> str:
    # 1st: st.secrets — works in Streamlit context
    try:
        return st.secrets[key]
    except Exception:
        pass
    # 2nd: os.environ — works when injected by main_ui.py
    value = os.environ.get(key)
    if value:
        return value
    # 3rd: .env file — works locally
    return os.getenv(key)
```

st.secrets[key]	Reads from Streamlit Cloud secrets dashboard. Only works inside Streamlit context.

os.environ.get(key)	Reads environment variables. main_ui.py injects secrets here before loading azure_services.py.
os.getenv(key)	Reads from .env file via python-dotenv load_dotenv(). Works locally.

main_ui.py — Why Secrets Are Injected First

The injection order is critical. Secrets must be in os.environ BEFORE azure_services.py loads.

```
# STEP 1: Inject secrets into os.environ FIRST
for key in ['AZURE_ENDPOINT', 'MODEL_DEPLOYMENT_NAME',
            'BING_CONNECTION_NAME', 'AZURE_API_KEY']:
    os.environ[key] = st.secrets[key]

# STEP 2: Load module AFTER secrets are injected
spec2.loader.exec_module(azure_mod)

# STEP 3: Create agent — now has all correct values
st.session_state.agent = TechAgent()
```

azure_services.py — Cloud vs Local Detection

We detect environment using the API key presence. No API key = local. API key present = cloud.

```
if self.AZURE_API_KEY:
    self.is_cloud = True
    self._setup_cloud()   # OpenAI SDK + api-key header
else:
    self.is_cloud = False
    self._setup_local()   # AIProjectClient + az login
```

_setup_cloud()	Uses OpenAI SDK with base_url pointing to /openai/v1. API key passed as api_key parameter. No agent management — TechGuru agent already exists on Azure.
_setup_local()	Uses AIProjectClient with DefaultAzureCredential. Creates fresh agent every run. Full Bing tool setup.
_chat_cloud()	Sends message history as input array to responses.create(). System prompt included as first message. No agent_reference — not supported on /openai/v1.
_chat_local()	Uses conversation ID for memory. Sends via openai_client.responses.create() with agent_reference pointing to TechGuru.

Streamlit session_state Explained

Streamlit reruns the entire script on every user interaction. session_state persists data across those reruns.

session_state.agent	Stores the TechAgent instance. Only created once. Survives all reruns.
session_state.messages	List of all chat messages. Each is a dict with role and content keys.
session_state.start_error	Stores any startup error so it can be displayed on screen.

```
# This only runs ONCE — first time page loads
if 'agent' not in st.session_state:
    st.session_state.agent = TechAgent()

# This runs EVERY rerun — redraws chat history
for message in st.session_state.messages:
    with st.chat_message(message['role']):
        st.markdown(message['content'])
```

SECTION 6 — GIT AND GITHUB WORKFLOW

Git and GitHub

Key Git Concepts

Repository (repo)	Your project folder tracked by Git. Contains all code and history.
Commit	A saved snapshot of your code at a point in time. Like a save point in a game.
Branch	A separate copy of your code. Work here without affecting main branch.
main branch	The live/production branch. Only merge here when code is working.
dev branch	Your working branch. Make all changes here first.
push	Upload your local commits to GitHub.
pull	Download changes from GitHub to your local machine.
merge	Combine changes from one branch into another.

First Time Setup (New Project)

```
git init                          # Start tracking folder
git add .                         # Stage all files
git commit -m 'Initial commit'    # Save snapshot
git remote add origin https://github.com/USERNAME/REPO.git
```

```
git branch -M main                # Rename to main
git push -u origin main           # Push to GitHub
```

Daily Workflow (Dev Branch)

```
git checkout dev                   # Switch to dev branch
# ... make your changes ...
git add .                          # Stage changes
git commit -m 'Description'        # Save snapshot
git push origin dev               # Push to GitHub
```

Deploying to Production (Merge to Main)

```
git checkout main                  # Switch to main
git pull origin main              # Get latest main
git merge dev                     # Merge dev into main
git push origin main              # Push — triggers Streamlit redeploy
git checkout dev                  # Go back to dev for next change
```

Fixing Push Rejection Error

If you see: 'rejected — fetch first', GitHub has changes you don't have locally.

```
git pull origin main --rebase      # Get remote changes first
git push origin main              # Now push works
```

Essential .gitignore

```
.env                      # Never expose API keys
.streamlit/secrets.toml # Never expose cloud secrets
__pycache__/              # Python cache — not needed
*.pyc                     # Compiled Python — not needed
.venv/                    # Virtual env — huge, not needed
create_permanent*.py      # Local utility scripts
test_*.py                 # Local test files
```

SECTION 7 — STREAMLIT CLOUD DEPLOYMENT

Deploying to Streamlit Cloud

Why Streamlit Cloud?

- Free hosting for public apps
- Connects directly to GitHub — auto-deploys on push to main
- Built-in secrets manager — keeps API keys safe
- Gives you a live URL like https://yourapp.streamlit.app

Step-by-Step Deployment

7. Push your code to GitHub (main branch)
8. Go to https://share.streamlit.io and sign in with GitHub
9. Click 'Create app'
10. Select your repository and set Main file to main_ui.py
11. Click 'Advanced settings' → 'Secrets' tab
12. Paste your secrets in TOML format (see below)
13. Click 'Deploy'

Streamlit Secrets Format

```
[secrets]
AZURE_ENDPOINT = 'https://your-resource.services.ai.azure.com/api/projects/your-project'
MODEL_DEPLOYMENT_NAME = 'gpt-4o-mini'
BING_CONNECTION_NAME = 'Bingsearch'
AZURE_API_KEY = 'your-actual-api-key-here'
```

💡 ⚠ **API key goes ONLY in Streamlit secrets — never in your code or .env file committed to GitHub.**

Authentication: Local vs Cloud

Local	Uses DefaultAzureCredential → reads your az login session. No API key needed in .env.
Streamlit Cloud	Uses OpenAI SDK with API key from secrets → calls /openai/v1 endpoint directly.
Why different?	azure-ai-projects SDK only supports Entra ID. API key only works on /openai/v1. Two environments need two approaches.
Detection method	If AZURE_API_KEY exists in environment → cloud mode. If not → local mode.

What Users See vs What You See

You (owner)	See Share, Edit, GitHub icons in top right corner of app.
Public users	See only the clean chat interface. No edit buttons. No icons.
Edit button	Opens your GitHub repo. Only visible to you when logged into Streamlit.
Private repo	Making GitHub repo private does NOT break the live app. Streamlit still deploys.

SECTION 8 — COMMON ERRORS AND FIXES

Common Errors Reference

Azure Errors

DefaultAzureCredential failed	Run 'az login' in terminal. Your Azure session expired.
Unauthorized — audience incorrect	API key being used where Entra ID is required. Use correct auth for each environment.
No value for given attribute	A config value is None. Check .env file has all required keys.
invalid_payload — model not present	Wrong parameter name. Use raw dict with 'kind': 'prompt' for create_version.
Insufficient privileges	Your Azure AD account cannot create Service Principals. Use API key approach instead.
Connection refused / 404	Wrong endpoint URL format. Should be: https://resource.services.ai.azure.com/api/projects/project-name

Python Import Errors

ModuleNotFoundError	Package not installed. Run: pip install package-name
Relative import error (..)	Use importlib.util.spec_from_file_location() for cross-platform imports instead of relative imports.
AttributeError: no attribute 'agent'	session_state.agent was never set — agent initialization failed. Check start_error for the real error.
No value for given attribute	A variable is None. Check that all os.environ keys are injected before module loads.

Git Errors

rejected — fetch first	GitHub has changes you don't have. Run: git pull origin main --rebase
Permission denied	SSH key not set up. Use HTTPS URL instead of SSH for remote.
Merge conflict	Two branches changed same file. Open file, manually fix conflict markers, then commit.
detached HEAD	Run: git checkout main — you're not on a branch.

Streamlit Errors

AttributeError: session_state has no attribute	Agent never initialized. Check start_error and display it with st.code()
Encoding errors (weird characters in output)	Save file as UTF-8. In VS Code: Ctrl+Shift+P → Change File Encoding → UTF-8
App crashes on refresh	Session state cleared. Ensure agent init is inside 'if agent not in session_state' block.
Secrets not loading	Inject secrets into os.environ in main_ui.py BEFORE loading azure_services.py via importlib.

SECTION 9 — SDK VERSIONS AND COMPATIBILITY

SDK Versions Reference

💡 ⚠ **azure-ai-projects has breaking changes between versions. Always check which portal version you are using.**

azure-ai-projects 1.x	For Foundry Classic (old portal). Uses from_connection_string(). Do NOT use with New Foundry.
azure-ai-projects 2.x	For New Foundry (toggle ON). Uses endpoint URL. Required for our project.
Install 2.x	pip install azure-ai-projects (installs latest 2.x)
Verify version	pip show azure-ai-projects
create_agent removed	Old method. New SDK uses create_version() with agent_name parameter.

threads removed	Old concept. New SDK uses conversations via openai_client.conversations.create()
from_connection_string removed	Old factory method. Use AIProjectClient(endpoint=..., credential=...) instead.

Key API Changes (1.x → 2.x)

Old (1.x)	New (2.x)
`AIProjectClient.from_connection_string()`	`AIProjectClient(endpoint=..., credential=...)`
`client.agents.create_agent()`	`client.agents.create_version(agent_name=..., definition=...)`
`client.agents.threads.create()`	`openai_client.conversations.create(items=[])`
`client.agents.messages.create()`	`openai_client.conversations.items.create()`
`client.agents.runs.create_and_process()`	`openai_client.responses.create(conversation=..., extra_body=...)`
`BingGroundingTool(connection_id=...)`	`BingGroundingTool(bing_grounding=BingGroundingSearchToolParameters(...))`
`PromptAgentDefinition(model=..., tools=...)`	Raw dict: `{'kind':'prompt','model':...,'instructions':...,'tools':[...]}`

SECTION 10 — QUICK REFERENCE CHEATSHEET

Quick Reference Cheatsheet

Commands I Use Every Day

```
az login                                # Authenticate with Azure locally
streamlit run main_ui.py                # Run app locally
git checkout dev                        # Switch to dev branch
git add . && git commit -m 'msg'        # Save changes
git push origin dev                     # Push to GitHub
git checkout main && git merge dev      # Deploy to production
git push origin main                    # Triggers Streamlit redeploy
pip show azure-ai-projects              # Check SDK version
python create_permanent_agent.py        # Recreate Azure agent
```

Files I Must Never Commit

- .env — contains Azure endpoint and local credentials
- .streamlit/secrets.toml — contains cloud API key

- create_permanent_agent.py — local utility with credentials
- test_*.py — local test files

When Things Break — Checklist

14. Is az login still valid? Run: az login
15. Are .env values correct? Run: python debug.py
16. Is TechGuru agent on Azure? Check ai.azure.com → Agents (New Foundry ON)
17. Are Streamlit secrets correct? Check manage app → secrets
18. Did secrets inject before module load? Check main_ui.py order
19. Is the right SDK version installed? Run: pip show azure-ai-projects

Environment Variables Needed

AZURE_ENDPOINT	https://your-resource.services.ai.azure.com/api/projects/your-project
MODEL_DEPLOYMENT_NAME	gpt-4o-mini (or your deployed model name)
BING_CONNECTION_NAME	Bingsearch (exact name from Connected Resources)
AZURE_API_KEY	CLOUD ONLY — from Azure portal Overview page. NOT in local .env

SECTION 11 — LESSONS LEARNED

Key Lessons From This Project

Things That Caught Us Out

- azure-ai-projects SDK does NOT support API key auth — only Entra ID. This is documented but easy to miss.
- New Foundry (2.x SDK) vs Classic Foundry (1.x SDK) are completely different. The toggle matters.
- Importing modules via importlib runs them immediately — secrets must be injected BEFORE this happens.
- session_state variables defined inside __init__ are local only — use self.variable to share across methods.
- A typo like 'instruction' vs 'instructions' causes silent failure — bot gets no system prompt.
- Streamlit Cloud HOME directory is /home/adminuser — but this is unreliable for environment detection. Use API key presence instead.

- Always test in small steps — never build the full app before testing each connection individually.

Professional Habits to Keep

- Always test connection first before building logic (test_connection.py pattern)
- Use step-by-step print statements to find exactly where code fails
- Keep credentials in .env locally and Streamlit secrets on cloud — never in code
- Use dev branch for development, main branch for production only
- Document as you build — future you will thank present you
- When stuck in an error loop — read the actual error message, don't just retry the same fix

You built a production-grade AI chatbot deployed on Azure AI Foundry with live web search, domain restriction, and cloud deployment. That is a real-world skill used by professional AI engineers.